Real-Time Object Detection for Autonomous Vehicles

Project Proposal



Real-time loop: every frame is processed and the results are drawn back onto the video

Figure 1. End-to-end flow of the proposed system: camera input, frame processing, detection, and display.

Team Leader: Hisham Mohammed

Team Members: Youssef Tamer, Mostafa Bayoumi, Gana Ahmed, Esraa Hussein

1. Introduction

Autonomous vehicles rely on computer vision and machine learning to understand their surroundings and make informed decisions. One of the most important capabilities of such systems is **real-time object detection**, which lets the vehicle identify and locate objects such as pedestrians, vehicles, traffic signs, and other obstacles in camera footage.

Detecting objects accurately in real driving environments is challenging, because road scenes vary significantly. **Lighting, weather, road type, object size, traffic density, and camera perspective** can all affect detection performance. The perception system must also process images quickly enough to support real-time operation.

This project develops a machine-learning-based object detection system that identifies important objects in road environments while keeping a suitable balance between **detection accuracy and processing speed**.

2. Project Overview

The project aims to build a computer vision and machine learning system that detects and classifies important road objects in real time. The system processes images or live video and identifies:

- Cars and other vehicles
- Pedestrians
- Traffic signs

- Road obstacles

For each detected object, the system displays a bounding box, the object class, and a confidence score. It is intended as a prototype for autonomous driving and advanced driver-assistance applications.

3. Problem Statement

Autonomous vehicles must understand their surroundings continuously and accurately. Failing to detect a pedestrian, vehicle, traffic sign, or obstacle can lead to unsafe decisions. Traditional computer vision techniques may struggle with complex scenes, changing lighting, crowded roads, and varied weather.

The project therefore aims to develop a machine-learning-based detector that can:

- Detect relevant road objects automatically.
- Classify the detected objects.
- Operate on live video.
- Provide predictions with low inference latency.
- Maintain reliable performance across environmental conditions.

4. Proposed Solution

The proposed solution is a **real-time computer vision system based on a deep-learning object detection model**. The system receives images or live video, processes each frame with a trained detector, and draws bounding boxes and class labels around the relevant objects.

The approach follows five stages:

4.1 Dataset collection and exploration

- Use a labeled dataset such as KITTI, COCO, or Open Images.
- Focus on pedestrians, cars, trucks/buses, motorcycles, bicycles, traffic signs, and obstacles.
- Analyze class distribution, image quality, resolution, and environmental conditions.

4.2 Data preprocessing

- Resize images to a suitable input size and normalize pixel data.
- Apply augmentation such as flipping, cropping, and rotation.
- Prepare the data for training and evaluation.

4.3 Object detection model

- Develop and train a detector, preferably a **YOLO-based architecture** because of its suitability for real-time applications.
- Use transfer learning and pretrained weights where appropriate.
- Fine-tune the model on the selected road-scene dataset.

4.4 Model evaluation

- Evaluate with **mAP, IoU, precision, recall, and FPS**.
- Compare detection accuracy against real-time requirements.
- Analyze errors such as missed objects and false detections.

4.5 Real-time deployment and MLOps

- Integrate the trained model with a camera or video stream and display detections with confidence scores.
- Test under daytime, low-light, urban, highway, and multi-object scenes.
- Track experiments and model versions with **MLflow** (or Kubeflow), monitor performance and speed, and support future retraining.

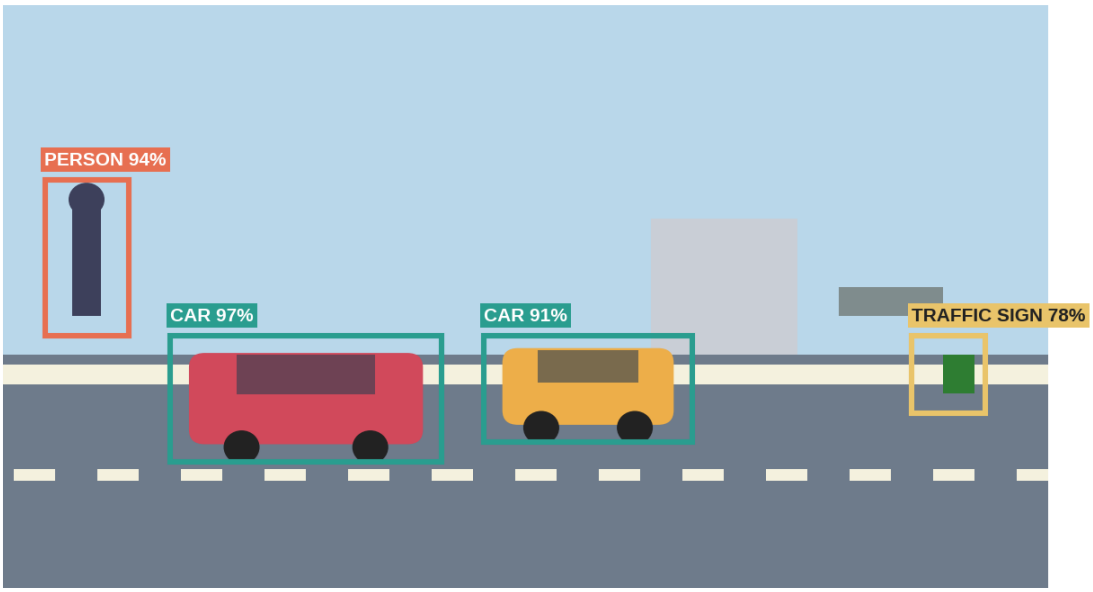


Figure 2. Illustrative output format (synthetic scene drawn for this proposal, not a real dataset frame): bounding boxes, class labels, and confidence scores.

5. Project Objectives

Primary objective: develop and deploy a real-time object detection model that identifies road objects from images and video streams.

Specific objectives:

- Collect and prepare an appropriate autonomous-driving dataset.
- Analyze the dataset to identify class imbalance and environmental variation.
- Preprocess and augment the images.
- Train an object detection model using transfer learning.
- Compare suitable object detection architectures.
- Optimize the model for real-time inference.
- Evaluate the model using mAP, IoU, precision, recall, and FPS.
- Integrate the model with a live camera or video stream.
- Test the system under different driving conditions.
- Implement experiment tracking and model monitoring.
- Prepare a final demonstration and technical report.

6. Project Scope

In scope	Out of scope
Image-based and video-based object detection Detection of selected road objects; bounding-box prediction; object classification Confidence-score visualization Dataset preprocessing, augmentation, and transfer learning Model evaluation, optimization, and experiment tracking Real-time camera/video testing Basic model monitoring Web/demo interface if time permits	Controlling an actual autonomous vehicle Making real driving decisions Controlling steering, braking, or acceleration Guaranteeing safety for real-world autonomous driving Replacing certified automotive safety systems Performing complete autonomous navigation Guaranteeing reliable performance in every weather or road condition

To keep the project achievable within the DEPI timeframe, the system is limited to perception and object detection.

7. Target Users and Stakeholders

Stakeholder	Interest
Instructor	Evaluate technical implementation and project deliverables
Project Team	Develop and demonstrate the system
ML/AI Engineers	Train and optimize the detection model
Computer Vision Developers	Handle video processing and detection visualization
End Users / Demonstrators	Interact with the real-time detection demo

8. Timeline and Milestones

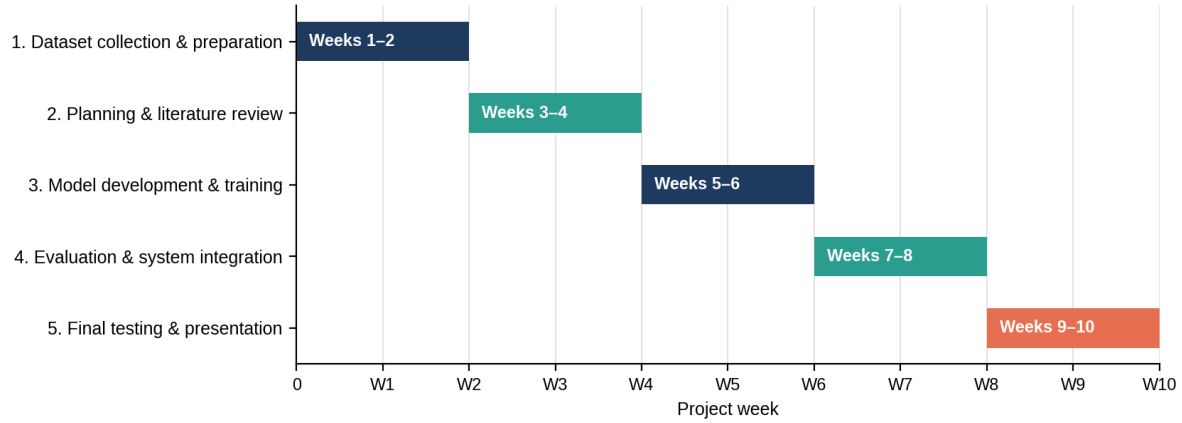


Figure 3. Ten-week project schedule by milestone.

Milestone	Timeline	Main activities	Deliverables
1. Dataset Collection & Preparation	Weeks 1–2	Select dataset (KITTI or COCO); inspect images, annotations, and classes; clean and preprocess; prepare splits; verify label formats.	Selected dataset, dataset description, prepared data
2. Planning & Literature Review	Weeks 3–4	Finalize objectives and scope; review YOLO, SSD, and Faster R-CNN; define requirements, KPIs, roles, timeline, and risks.	Project plan, literature review, requirements specification, risk assessment
3. Model Development & Training	Weeks 5–6	Configure the model; establish a pretrained baseline; fine-tune; run initial experiments and record results.	Trained model, training logs, initial results
4. Evaluation & System Integration	Weeks 7–8	Evaluate precision, recall, mAP, IoU, FPS, and latency; integrate with OpenCV video processing; test on road videos.	Evaluation report, integrated system, initial video demo
5. Final Testing & Presentation	Weeks 9–10	Test road scenarios; fix issues; finalize documentation; prepare slides; demonstrate the system.	Final report, test results, presentation, final demo

9. Technology Stack

Area	Technology
Programming	Python
Deep learning	PyTorch / TensorFlow
Object detection	YOLO
Computer vision	OpenCV
Data processing	NumPy, Pandas
Visualization	Matplotlib
Experiment tracking	MLflow
API / backend	Flask or FastAPI
Model optimization	ONNX
Version control	Git / GitHub
Dataset	KITTI / COCO / Open Images or suitable alternative
Deployment	Local machine; Azure if feasible

10. Methodology

The project follows an iterative machine-learning process:

Planning → Data Collection → Data Preparation → Model Development → Evaluation → Optimization → Deployment → Testing → Monitoring → Final Presentation

Rather than waiting for a finished model, the team evaluates intermediate results continuously and improves the system as it goes.

11. Team Roles and Responsibilities

Member	Role	Main responsibilities
Hisham Mohammed	Project Manager / ML Lead	Planning, coordination, model strategy, deadlines
Youssef Tamer	Data Engineer	Dataset collection, cleaning, preprocessing, EDA
Gana Ahmed	ML Engineer	Model training, tuning, evaluation
Mostafa Bayoumi	Computer Vision / Deployment Engineer	OpenCV, real-time video, model integration
Esraa Hussein	MLOps / Backend & Documentation	MLflow, deployment, monitoring, API, documentation

12. Risk Assessment and Mitigation

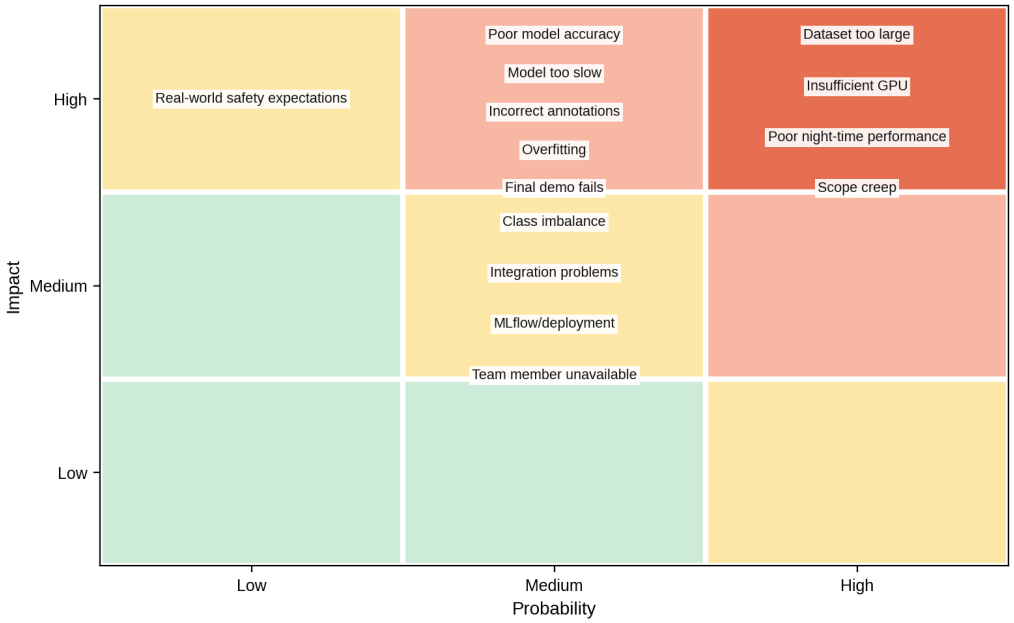


Figure 4. Risk matrix based on the risk register below (probability vs. impact).

Risk	Probability	Impact	Mitigation
Dataset is too large	High	High	Use a representative subset initially
Insufficient GPU resources	High	High	Use pretrained models and cloud/Colab resources
Poor model accuracy	Medium	High	Improve preprocessing, augmentation, and transfer learning
Class imbalance	Medium	Medium	Augmentation, resampling, or class balancing
Model is too slow	Medium	High	Use a lightweight YOLO model and optimize inference
Poor night-time performance	High	High	Include diverse lighting conditions in training
Incorrect annotations	Medium	High	Inspect and validate annotations
Overfitting	Medium	High	Augmentation, validation, and early stopping
Integration problems	Medium	Medium	Build the real-time pipeline early
MLflow/deployment problems	Medium	Medium	Keep a simpler local tracking/deployment fallback
Team member unavailable	Medium	Medium	Document work and maintain a shared repository
Scope becomes too large	High	High	Prioritize detection before advanced features
Final demo fails	Medium	High	Prepare prerecorded videos as backup
Real-world safety expectations	Low	High	Define the system clearly as a prototype, not a production controller

Highest-priority risks

- **Insufficient computing resources.** Start with pretrained lightweight models and use transfer learning rather than training from scratch.

- **Real-time performance.** A highly accurate model may be too slow for live video. Optimize for both accuracy and FPS.
- **Environmental variation.** Use diverse training data and augmentation, then explicitly test different scenarios.
- **Scope creep.** Limit the project to perception and object detection. Do not attempt steering, navigation, braking, or full autonomous decision-making.

13. Key Performance Indicators

KPIs focus on ML performance and real-time performance rather than generic measures such as user adoption.

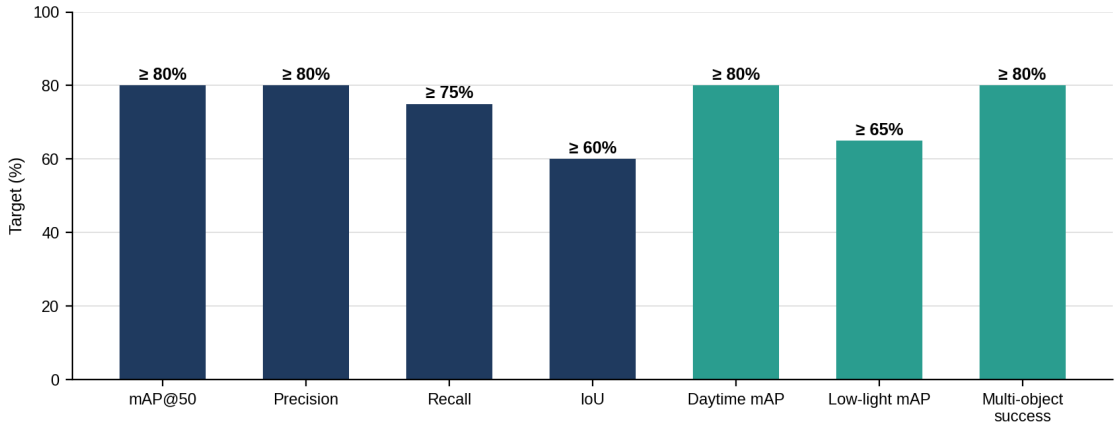


Figure 5. Main accuracy and robustness targets (project goals, not guaranteed outcomes).

13.1 Model accuracy

KPI	Target
mAP@50	≥ 80%
Precision	≥ 80%
Recall	≥ 75%
IoU	≥ 60%
False detection rate	≤ 20%

13.2 Real-time performance

KPI	Target
FPS	≥ 20 FPS
Average inference latency	≤ 100 ms
Detection response time	≤ 150 ms
System startup success	≥ 95%

13.3 Robustness

KPI	Target
Daytime detection mAP	≥ 80%
Low-light detection mAP	≥ 65%
Multiple-object detection success	≥ 80%
Performance degradation across environments	≤ 20%

13.4 Project management

KPI	Target
Milestones completed on time	≥ 90%
Assigned tasks completed	≥ 95%
Team meeting attendance	≥ 90%
Required deliverables completed	100%
Critical unresolved issues before final demo	0

13.5 MLOps

KPI	Target
Experiments tracked	100%
Final model version documented	100%
Deployment reproducibility	≥ 95%
Monitoring availability	≥ 95%
Retraining procedure documented	100%

14. Requirements

14.1 Functional requirements

- **FR-01** The system shall accept a supported image or video file as input.
- **FR-02** The system shall detect the selected object classes supported by the trained model.
- **FR-03** For each detection, the system shall display a bounding box, predicted class label, and confidence score.
- **FR-04** The system shall process video frame by frame and display detections continuously.
- **FR-05** The operator shall be able to configure the confidence threshold when supported.
- **FR-06** Annotated results shall be saveable to an output file or folder.
- **FR-07** The evaluation workflow shall record mAP and selected precision, recall, IoU, FPS, and latency.
- **FR-08** The team shall record the dataset, class mapping, model version, and main training and inference settings.
- **FR-09** The system shall show an understandable error when an input cannot be opened or has an unsupported format.
- **FR-10** The project shall provide setup and run instructions sufficient to reproduce the baseline demo.

14.2 Non-functional requirements

- **NFR-01 Performance:** measure and report FPS and per-frame latency on declared hardware, recording image size and model settings.
- **NFR-02 Detection quality:** evaluate on a held-out test set with a stated mAP/IoU protocol, reporting precision and recall.
- **NFR-03 Usability:** a user can run the demo from documented steps and read detections from labels and boxes.
- **NFR-04 Reliability:** missing, unreadable, or unsupported inputs are handled without an unhandled crash where feasible.
- **NFR-05 Maintainability:** code is organized into understandable modules, with dependencies and configuration documented.
- **NFR-06 Reproducibility:** record dataset source and version, class mapping, random seeds, model weights, and settings.
- **NFR-07 Security and privacy:** no credentials or secrets in the repository; avoid unnecessary personal data; respect dataset terms.
- **NFR-08 Portability:** document Python and library versions, GPU requirements, and CPU-fallback limitations.
- **NFR-09 Safety limitation:** present the system as an educational perception prototype, not a certified or safe vehicle-control system.

14.3 Constraints and assumptions

- The project is limited by available hardware, dataset licensing and availability, and the team's experience.
- Only publicly accessible datasets are used, in accordance with their terms; preprocessing and label conversion are documented.

- The initial target is a local demonstration using images or prerecorded video. Live-camera support and a web interface depend on time and successful integration.
- The detector recognizes only the classes in its trained or pretrained label set; unsupported classes are not claimed without additional work.

15. Related Projects and Solutions

Udacity CarND Vehicle Detection

An educational project that detects vehicles in road images and video using traditional computer vision: HOG features, an SVM classifier, a sliding window, and frame heatmaps to reduce false detections.

Strengths: A clear, easy-to-follow pipeline that demonstrates vehicle detection and tracking across frames.

Limitations: Focuses on vehicles only and uses hand-crafted features rather than a modern deep-learning detector.

Difference from our project: Our project covers several road-object categories with a deep-learning detector.

Source: https://github.com/udacity/CarND-Vehicle-Detection

Baidu Apollo Perception

An open-source autonomous-driving platform whose architecture supports perception as one part of a larger system.

Strengths: An established platform with many autonomous-driving components that supports integrating perception into a full system.

Limitations: Broad scope and integration requirements make it more complex than a standalone detector; full reproduction may need extra software, sensors, and hardware.

Difference from our project: We focus on implementing and evaluating one road-scene detection module rather than a full driving platform.

Source: https://apollo.baidu.com/docs/apollo/latest/index.html

Autoware

An open-source autonomous-driving framework built on ROS, with modules for sensing, perception, localization, planning, and vehicle control.

Strengths: Modular design lets components work together and supports complete driving applications.

Limitations: Setting up and integrating the full stack is demanding for a small project and exceeds the needs of object detection alone.

Difference from our project: We concentrate on detecting road objects from camera input and measuring detection performance.

Source: https://autoware.org/autoware-overview/

Ultralytics YOLOv8

An open-source family of computer-vision models supporting detection and other tasks, with tools for training, validation, inference, and deployment.

Strengths: A practical training and inference workflow, pretrained models, and an efficient starting point for detection.

Limitations: General-purpose pretrained weights may not meet road-specific requirements; fine-tuning and evaluation on the chosen dataset are needed.

Difference from our project: We use YOLOv8 as the starting architecture and adapt and evaluate it for road-object categories using mAP, FPS, and latency.

Source: https://docs.ultralytics.com/models/yolov8

Conclusion

The reviewed solutions show different approaches to autonomous-driving perception. Udacity CarND offers an understandable traditional pipeline, while Apollo and Autoware provide broader frameworks. YOLOv8 is a practical foundation for a standalone deep-learning detector.

Based on this review, the project will implement and evaluate a real-time road-object detection solution on a suitable driving dataset, investigating how accuracy and processing speed meet the project's requirements.

16. Expected Final Product

The final system receives live camera or video input and produces annotated output:

Camera → Video Frame → Object Detection Model → Detected Objects → Bounding Boxes + Labels + Confidence → Display

17. Success Criteria

The project is successful if the team can demonstrate that:

- The system detects the selected road objects correctly.
- The trained model achieves acceptable mAP and IoU.
- The system runs at an acceptable real-time FPS.
- The model can process live video.
- Bounding boxes, labels, and confidence scores are displayed correctly.
- The model performs reasonably under different environmental conditions.
- Experiments and model versions are properly tracked.